Ders 2 Genişletilmiş Notu: Yazılım Geliştirme Süreç Modelleri, Çevik Yaklaşım, XP ve Scrum

Ders 2 Genişletilmiş Notu: Yazılım Geliştirme Süreç Modelleri, Çevik Yaklaşım, XP ve Scrum

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Geçen Dersin Tekrarı
- 2. Yazılım Mühendisliğinin Beslendiği Alanlar ve Yazılım Sistemi
- 3. Dersin Hedefleri
- 4. Şelale (Waterfall) Modeli
- 5. Prototip Model
- 6. Artımlı (Incremental) Model
- 7. Spiral Model
- 8. RAD (Rapid Application Development) Modeli
- 9. Bileşen Tabanlı (Component-Based) Geliştirme
- 10. Birleşik Süreç (Unified Process)
- 11. Çevik (Agile) Yaklaşım
- 12. Scrum ve Diğer Modellerin Karşılaştırılması
- 13. Uç Programlama (XP, Extreme Programming)
- 14. Scrum
- 15. Model Seçimi ve Kuruma Özgü Uyarlama

1. Geçen Dersin Tekrarı

Hoca dersin bu dönemki ikinci dersi olduğunu söyleyerek, geçen hafta katılanlar için bir tekrar, katılmayanlar için de bir başlangıç yaptı.

1.1 Kaynaklar ve Ders İçeriği

Dersin temel kaynakları, yazılım mühendisliğinin klasik ders kitaplarıdır: Pressman'ın ve Sommerville'in *Software Engineering* kitapları. Bu derste projeler nesneye yönelik (object-oriented) yazılım geliştirme ile yapılacağı için Larman'ın UML uygulaması üzerine olan kitabı da faydalıdır. Hoca, nesneye yönelik kavramlar ve UML çizimiyle ilgili başka kaynaklar da bulunabileceğini, ama bunların bütün dünyanın kullandığı temel klasik kitaplar olduğunu söyledi.

Ders içeriği şöyle akar: Geçen hafta yazılım mühendisliğine giriş yapıldı. Bu hafta süreç modelleri (process models) işlenecek. Süreç modelleri daha çok Pressman'ın kitabından derlendi. Sonra gereksinim mühendisliği (requirements engineering) gelecek, yani analiz aşamasının daha ayrıntılı ifadesi. Analiz aşamasının örneği olarak nesneye yönelik çözümleme görülecek. Ardından tasarım, test teknikleri, proje yönetimine giriş ve yazılım ölçütleri gelecek. Yazılım proje yönetimi kapsamında risk yönetimi ve kalite ile ilgili kavramlar da görülecek: kaliteli yazılım deyince ne anlıyoruz, hangi özellikleri vardır. Hocaya göre dersin hedefi, yazılım geliştirme sürecindeki bütün adımları görmek ve bunları en teknik şekilde incelemektir.

1.2 Yazılım Yalnızca Program Değildir

Hoca, yazılım deyince programdan başka neler düşünüldüğünü sordu. Öğrencilerden gelen cevap dokümantasyon, yani belgeler oldu. Hoca bunu açtı: Yazılım şunların hepsinden oluşur:

- Yazılımı kullanıcılara anlatan, nasıl kullanacaklarını gösteren kullanıcı belgeleri.
- Daha sonra düzeltme ve bakım yapacak kişilere iç yapı ve veri yapısı hakkında bilgi veren belgeler.
- Yazılımın yapım aşamasını anlatan belgeler.
- Veri tabanının nasıl ve hangi test verisiyle test edildiğini anlatan belgeler.

Bunların hepsi birlikte yazılımı oluşturur. Yani bunlar yazılımın elemanlarıdır.

1.3 Yazılım Türleri

Geçen hafta görülen yazılım türleri hatırlatıldı: sistem yazılımı, büyük hacimli verilerle uğraşan mühendislik yazılımları, gömülü (embedded) yazılımlar ve uygulama yazılımları.

- **Ürün hattı (product line):** Bir yazılım evinin, sürekli aynı altyapıyla ve konuya yönelik bir dille geliştirdiği yazılım ailesidir.
- **Hazır yazılımlar (COTS, commercial off-the-shelf):** Paket programlar. Bunlar sistem yazılımı, uygulama yazılımı ya da müşteriler tarafından kullanılacak genel amaçlı yazılımlar olabilir.
- **Kurumsal yazılımlar:** Kurumların temel işlemlerini gerçekleştirdiği yazılımlar. Hoca bunun iş hayatında sık rastlanacak bir terim olduğunu söyledi.
- **Eski yazılım (legacy software):** İş sürecinin önemli bir parçası olan, uzun süredir kullanılan yazılımdır.

Eski yazılımın olumsuz yanları şunlardır: Dokümantasyonu eksik olabilir. Hep yamalar yapıldığı için zamanla kodu karmaşıklaşır. Esnek değildir. Donanımla sıkı bir ilişkisi vardır. Yazılım mühendisliğindeki gelişmelerle, ekip çalışmasıyla üretilen yeni yazılımlar kurallara daha uygun geliştirilir, ama eski yazılımlarda bu konuda eksiklikler olabilir. Bunlarda bazen değişiklik yapılır, bazen de kurumun çok temel işlerini yaptığı için yamalarla idare edilir.

1.4 Yazılım Hakkında Yanılgılar

Hoca yazılım hakkındaki yanılgıları üç taraftan tekrar etti: müşteri, yönetim ve geliştirici.

Müşterinin yanılgıları:

- "Programın yazılmasına başlamak için amaçları genel olarak belirlemek yeter, ayrıntılar sonra kararlaştırılır." Hocaya göre analizde ilk başta temel şeyler, basit bir anlatımla verilebilir. Ama müşterinin isteklerini tasarıma ve kodlamaya geçmeden belirtmesi gerekir. Gereksinimlerin belirli olması şarttır.
- "Yazılım esnektir, istediğim anda değişiklik yaptırabilirim." Bu müşterinin çok temel bir yanılgısıdır. Yazılım yaşam döngüsü ilerledikçe isteklerin yazılıma uyarlanmasının bedeli üstel olarak artar.

Yönetimin yanılgıları:

- "Ekibi ilk başta minimum kuralım, işler yetişmezse yeni kişiler ekleriz." Yeni kişiler eklendiğinde karmaşıklık üstel olarak artar. Bunun planlı yapılması gerekir.
- "Geliştirilmesini üstlendiğim yazılımı tamamen ya da kısmen fason yaptırırım." Bazı işler başka firmalara verilebilir, ama bu başta planlanmalı ve yoğunluk kurumda kalmalıdır. Proje ilerlemesini kendi içinde denetleyemeyen firma, dışarıya verdiği işi izlemekte zorlanır. İçeride izlemeyi iyi yapacak sağlam bir ekip olmalıdır. İşi takibini yapamayacak şekilde başka bir firmaya havale etmek doğru değildir.
- "Açık kaynak yazılım üretirsem kâr edemem." Açık kaynak yazılım, danışmanlık hizmetleri ve açık kaynakla ilgili farklı iş modelleriyle kâr ettirir.

Geliştiricinin yanılgıları:

- "Yazılımı tamamlayıp müşteriye teslim edince işim biter. Ben kodu ve algoritmayı iyi yazmakla sorumluyum." Oysa yazılıma harcanan çabanın en az yarısı teslimden sonra harca-

nır. Takip bırakılmamalıdır: Firma kurulumu tamamladı mı, bizim oluşturduğumuz ortamı kurabildi mi, veri arttığında sorun çıkıyor mu?
- "Yazılımı tamamlamadan kalitesini ölçemem." Kalite ölçümü için geliştirme sürecinin tamamında ölçüm yöntemleri vardır. Yazılım, kaliteyi ihmal etmeden, her adımda gözden geçirme yapılarak geliştirilmelidir.
- "Yazılım mühendisliği gerekleri vaktimi alıyor. Programı yazarım, çalışır, en önemlisi budur." Çalışan yazılım tabii ki önemlidir, ama bütünü olmalıdır: gereksinimlerin ne olduğu, raporlar ve yaşam döngüsünün sonunda karşılaşılacak şeyler baştan kurallı olarak ele alınmalıdır.

Hoca bu yanılgılardan şu dersleri çıkardı: Yazılım program demek değildir. Gereksinim analizi önemlidir. Dokümantasyon ve sınama (test) önemlidir. Geliştirme sürecindeki adımlar takip edilmelidir. Hocaya göre yazılım projelerinin hâlâ yüzde ellisi başarısızlığa uğruyor.

1.5 Mühendislik ve Yazılım Mühendisliği Tanımı

Hoca mühendisliğin hendese ile ilişkisini hatırlattı. Mühendislik çalışmalarının iki ön şartı vardır: ölçme ve tasarım. Yapılan şeyin kalitesi mutlaka ölçülmeli, gerçeklemeye başlamadan önce de tasarım yapılmalıdır. Yazılım mühendisliğinde de büyüklük, karmaşıklık gibi aritmetiksel ölçümler yapılır. Ne istendiği, ne yapılacağı ve modelin nasıl oluşturulacağı belirlendikten sonra kod oluşturulur.

Geçen hafta verilen tanım şuydu: Yazılım mühendisliği, kaliteli yazılımı en ekonomik yolla oluşturmak üzere mühendislik prensiplerinin kullanıldığı yazılım geliştirme çalışmalarına önderlik eder. Hoca bu tanımdan dersin yol haritasını çıkardı:

- **Kaliteli yazılım:** Önce kaliteli yazılımın ne olduğunu anlamalıyız.
- **Ekonomik yol:** Yazılım projelerini nasıl takip edeceğimizi, planı nasıl oluşturacağımızı, maliyet tahminini nasıl yapacağımızı düşünmeliyiz.
- **Mühendislik prensipleri:** Tasarımı ve ölçmeyi sürece nasıl yerleştireceğimizi bilmeliyiz.

1.6 Temel Aktiviteler ve Nesneye Yönelik Akış

Yazılım geliştirme sürecinin temel adımları (temel aktiviteler) şunlardır: gereksinim analizi, tasarım, gerçekleme (implementation) ve test. Doğrudan kod yazarak başlanamaz.

Gereksinim analizinde hedef "ne yapacağız?" sorusuna cevap almaktır. Kurumsal bir yazılım için bu soru, işletmedeki kullanıcıların ihtiyaçlarının ne olduğudur. Kullanıcıların yaptığı işi yazılım aracılığıyla gerçekleştirmek istiyoruz. O halde önce iş sürecini anlamalıyız: Önce ne yapılıyor, sonra ne yapılıyor? Hangi departman hangisine bilgi gönderiyor? Hangisi hangi raporu istiyor? İşi iş sırasıyla anladıktan sonra bilgisayar ortamına nasıl aktarılacağını tasarlarız. Bunu anlamamıza yardımcı olan modellemeler ve diyagramlar vardır.

Nesneye yönelik düşünüldüğünde akış şöyledir:

1. Gereksinim analizinde kullanıcı hikâyeleri (user stories) alınır ve use case diyagramlarıyla kurallı biçimde yazılır. Sıraları belirlenir, işler ilişkilendirilir. Nasıl bir veri saklama gerektiği anlaşılır.
2. Anlaşılanlar tasarlanır, sınıf diyagramları ayrıntılı olarak oluşturulur.
3. Veri tabanına geçerken, veri tabanını yaratmadan önce ER diyagramları çıkarılır.
4. Tasarım modeli oluşturulur. Klasik süreçte burada yapı diyagramı kullanılırdı, ama bu derste hep nesneye yönelik yaklaşım kullanılacak. Analizde başlanan sıralama (ardışıl) diyagramları burada daha ayrıntılı çizilir. Kontrol akışı için durum diyagramları oluşturulur.
5. Bunlara bağlı olarak kod yazılır, veri tabanı ve tablolar yaratılır.
6. İstenilenlerin doğru yapılıp yapılmadığı test edilir.

   Hocanın vurgusu: Bunları anlatırken hep sırayla anlatıyoruz, ama gerçekte adımlar böyle sırayla gitmez. Süreç modellerinde bir ardışıklık olmak zorunda değildir. Adımların nasıl dizildiği, hangi modelin uygulandığını belirler.

1.7 Şemsiye Aktiviteleri

Pressman'ın kitabında yazılım geliştirmede şemsiye aktiviteleri (umbrella activities) olarak tanımlanan aktiviteler vardır. Hoca bunlara destek aktiviteleri diyor. Bunlar yazılım geliştirmenin temel işlemlerini koruyarak destekler: proje planlama, kalite yönetimi, yazılım ölçümü, risk analizi ve yazılım konfigürasyon yönetimi. Hoca bu adımları sektörde de yakından göreceğinizi söyledi. Bu aktiviteler de süreç modellerinin içine girmiştir. Süreç modelleri genellikle bunların hangi sırayla yapılacağını da tarif eder.

1.8 CASE

Hoca CASE (Computer-Aided Software Engineering) kavramını tekrar etti. CAD/CAM nasıl tasarım ve üretime destek olan yazılımlarsa, CASE de yazılım mühendisliği süreçlerini destekleyen yazılımlardır. Yazılım geliştirme sürecindeki esas adımlar analiz, tasarım, gerçekleştirme ve testtir. CASE bunları destekler. Proje yönetimi, kalite yönetimi, konfigürasyon yönetimi ve risk yönetimi gibi destek aktivitelerini destekleyen yazılımlar da vardır. Günümüzde bunların hepsini sırayla destekleyen platformlar bulunur ve birçok firmada ekip çalışmaları bunlar aracılığıyla yönetilir.

Bütünleşik bir CASE şu bileşenlerden oluşur:

- Bir grafik arayüz.
- Veri modellerini, veri sözlüğünü ve işlemleri tanıtan bir veri tabanı.
- Analiz ve tasarımdaki diyagramları oluşturmak için diyagram çizim araçları.
- Tasarım ve analizi ilişkilendirerek gerçekleştirme yeteneği. Hoca proje planındaki işlerin de bu şekilde gerçekleştirilebileceğini söyledi.
- Kod üreticiler ve kod çeviriciler. Örneğin bir tasarım tanımlanır ve bunun kodunun belirli bir dilde yazılması istenir. Otomatik kod geliştirme araçları da bu kapsamdadır.

2. Yazılım Mühendisliğinin Beslendiği Alanlar ve Yazılım Sistemi

Hoca yazılım mühendisliği tanımına başka bir yaklaşım daha getirdi. Yazılım mühendisliği şu bilim dallarından faydalanır:

- **Bilgisayar bilimi:** İyi bir yazılım üretmek için bilgisayar altyapısını bilmek gerekir.
- **Yönetim bilimi:** Proje yönetimi ve kalite yönetimi için temeldir.
- **Ekonomi:** Yazılım mühendisliği ilk olarak yazılım ekonomisi olarak başlamıştır. Profesyonel yazılım evleri kurulduğundan beri, bir proje başlamadan önce "bu ne kadara mal olacak?" sorusu sorulur. Hoca, sizden biri amatörce bir yazılım istese bile bu sorunun sorulabileceğini söyledi. Maliyeti belirlemek için ekonomiyle ilişki kurmak, Sistem Analizi dersinde görülen ekonomik fizibiliteyi yapabilmek gerekir.
- **Bilişim:** Sorun çözümünde mühendislik yöntemi kullanılır.

Bu noktadan başlanarak bugünkü tanıma gelinmiştir: Kaliteli yazılımı en ekonomik yolla hazırlamak üzere mühendislik yöntemlerinin kullanılması yazılım mühendisliğidir.

Yazılım sistemi (software system) tek başına görülemez. Bir yazılım hazırlanacağı zaman şunlar bilinmelidir:

- Yazılım hangi kurumda kullanılacak? Kurum nasıl bir ortamda, iç özellikleri neler?
- Kurumdaki diğer bilgi sistemleri neler? Diğer uygulamalar hangi bilgi sistemleriyle yürütülüyor?
- Hangi donanım kullanılıyor, nasıl bir bilgisayar altyapısı var?

Bunlar bilinirse istenilen yazılımı üretmek mümkün olur.

3. Dersin Hedefleri

Hoca, ders bittiğinde öğrencilerin şunları bilmesini istediğini söyledi:

- Yazılım mühendisliği ile programcılık arasındaki fark. Yazılım nedir, yazılım nasıl geliştirilir, geliştirme süreci nedir.
- Yazılım geliştirme sürecindeki modeller.
- Yazılım yeterlilik olgunluk modeli, yani yazılım sertifikasyonu.

- Yazılım standartları. Hoca, nasıl ağ (network) standartları varsa yazılım geliştirirken de uyulması gereken standartlar olduğunu söyledi.
- CASE teknolojisi.
- Yazılım proje maliyeti ve maliyet tahmin yöntemleri, yazılım ölçüm kriterleri, yazılım proje planının nasıl hazırlanacağı.
- Kalite özellikleri ve kalite yönetim teknikleri.
- Yazılım bakımı (software maintenance).
- Yazılım konfigürasyon yönetimi (Software Configuration Management).
- Gereksinim analizi, gereksinim belirtimi (specification), tasarım süreci ve tasarım gösterimleri (notation). Bunların hepsinin nesneye yönelik olarak görülmesi.
- Tasarım kalıpları (design patterns).

Yazılım konfigürasyonu ve konfigürasyon yönetimi: Yazılım, program artı diğer elemanlardır. Bu elemanların hepsinin ve yazılımla aralarındaki ilişkilerin tanımlanması, o yazılımın konfigürasyonudur. Konfigürasyon yönetimi ise bu elemanların yazılımın kullanım ve geliştirme süreci boyunca nasıl değiştiğinin, hangi evrede nasıl bir yapıda olduğunun takibidir. Sürüm kontrolü (version control) de bunun içindedir.

Hoca bir konuyu özellikle belirtti: Yazılım Mühendisliği, bazı dersler alındıktan sonra alınan bir derstir. Hoca hep Sistem Analizi ve Tasarımı dersine referans veriyor. Mühendislik tamamlama ya da yüksek lisans hazırlıktaki öğrenciler için bu ilk ders olabilir. Bu durumda atıf yapılan derslerin kaynaklarına bakabilecek durumda olmak gerekir. Hangi UML diyagramının nasıl çizildiği ve nesneye yönelik kavramlar bu derste bilindiği varsayılarak verilecek. Nesneye Yönelik Programlama dersinde nesne kavramı ve nesne yapısı görülür, use case'ler pek görülmez. Bu derste ise UML diyagramlarıyla tasarım modelinin nasıl çıkarıldığı uygulanacak, bütün olarak bir hatırlatma yapılacak ve tasarım kalıpları görülecek. Hedef yalnız nesneye yönelik kavramları anlatmak değildir, tasarım süreci üzerinde de durulur.

4. Şelale (Waterfall) Modeli

Klasik sürecin adı şelaledir (waterfall). Aynı model için kullanılan başka adlar da vardır: ardışıl süreç, lineer akış, doğrusal model. Hocaya göre bu adların hepsi modelin çalışma biçimini anlatır.

Adımlar şöyledir:

1. Gereksinim analizi.
2. Tasarım.
3. Gerçekleme (implementation). Burada mühendislik testi denen birim testi ve bütünlük testi de yapılır.
4. Bütünleştirme (integration). Bir kullanıcı testi yapılır.
5. Kullanıma hazırlık (release) ve teslim (deployment).
6. İşletim ve bakım. Ürünün kullanım aşamasındaki işletimi ve bakımı da sürecin bir parçasıdır.

Kullanıcıdan alınan bilgiler bu adımlarla işlenerek ürün haline gelir. Bir adım bittiğinde diğerine geçilir. Gereksinim analizindeki bütün işlemler tamamlanınca bir gereksinim analiz raporu hazırlanır ve tasarıma öyle geçilir. Tasarım bitince tasarım raporu hazırlanır ve gerçeklemeye geçilir. Gerçekleme kodlama, test ve veri tabanının hazırlanmasıdır. Sonra veri tabanı yaratılır, sonra kurulum yapılır. Hepsi sırayla gerçekleşir.

Yine de bir sonraki adıma geçildiğinde sorun çıkarsa önceki adıma dönüp sorunu çözme imkânı vardır. Gereksinim analizinde bir hata görülürse kullanıcıya kadar geri dönüp güncel bilgileri almak da mümkündür.

> **Hocanın vurgusu:** Şelale modelinin dezavantajı, ürünün kullanıcıya verilene kadar ürün halinde görülememesidir. Müşteriye hep raporlar, şekiller, modeller gider, ama çalışan ürünü en sonunda görür. Kullanıcının kullanacağı ürünü zihninde canlandırması zaman alır. İsteklerini ancak en son ürünü çalıştırıp fark ettiğinde verebilir. Adımlar hep birbirini bekler.

5. Prototip Model

Şelale modelini biraz hızlandırmak için prototip model önerilmiştir. Bu modelde analiz, tasarım ve gerçekleme müşteriye sürekli sorularak, daha kolay yazılım araçlarıyla yapılır ve yazılımın bir prototipi oluşturulur. Prototip üzerinde konuşulur. Prototipte anlaşma sağlanınca asıl sistemin analizi, tasarımı ve bütünleştirmesi yapılır ve ürün ortaya çıkar. Hoca bunun da aslında klasik yazılım geliştirme süreçlerinden biri olduğunu söyledi.

6. Artımlı (Incremental) Model

Artımlı yazılım geliştirmede amaç, yazılımı müşterinin kullanımına daha hızlı sunmaktır. Yazılım bir çekirdek ve etrafındaki modüller halinde düzenlenir. Bittikçe teslim edilen ve sürekli geliştirilen bir yazılım olarak gerçekleştirilir. İterasyon 1, iterasyon 2, iterasyon 3 diye ilerlenir. Her iterasyon sonunda yeni bir artım (increment), yani yeni bir yazılım ürünü elde edilir.

Hocanın örneği bir öğrenci işleri yazılımıydı. Bu çok büyük bir yazılımdır, ama üniversiteye yeni giren öğrenciler vardır ve en azından onların bilgilerinin alınıp sisteme konması gerekir.

1. Önce yeni giren öğrencinin bilgilerinin alınıp veri tabanına aktarıldığı ilk modül oluşturulur.
2. Sistem ilerledikçe transkript verme gibi ek yazılımlar ve başka belgelerin oluşturulması eklenir.
3. Sonraki artımlarda çevre modüller eklenerek yazılım tamamlanır.

Bunun başta planlanması gerekir: İlk öncelik hangi modüle verilmeli? Sonraki eklemeler hangi modüllerle yapılmalı? Artımlar önceden planlanırsa ve müşteri yazılımı hızlı kullanmak istiyorsa bu yola gidilebilir. Ama hocaya göre her projede geçerli değildir.

7. Spiral Model

Artımlı model üzerinde biraz daha çalışılarak, hem iteratif hem de evrimsel bir yapı kazandırmak için spiral model önerilmiştir. Spiral modelde hem dikey boyutlar hem de evrimsel bir yapı vardır.

7.1 Çevrimler

Hoca aranın ardından spiral modeli daha iyi gösteren ikinci bir şekil açtı. Şekilde iç içe çevrimler vardı:

- **En içteki (koyu renkli) çevrim:** Prototipin geliştirilmesi için.
- **İkinci çevrim:** Prototipin üzerine eklemeler yapılarak, kullanıma alınacak asıl ürünün geliştirilmesi için.
- **Sonraki çevrim:** Ürünü zenginleştirmek, ürüne eklemeler yapmak için.
- **Sonraki boş çevrim:** Bakım aşamasında da aynı adımların takip edileceğini gösterir.

7.2 Her Çevrimdeki Dikey Bölümler

Her çevrimde aynı dikey bölümlerden geçilir:

1. **Müşteriyle iletişim:** Müşteri dinlenir, başlangıç gereksinimleri ve kullanıcı değerlendirmeleri alınır.
2. **Planlama.**
3. **Risk analizi:** Her adımda, prototipte bile bir risk analizi vardır.
4. **Mühendislik:** Analiz ve özellikle tasarım.
5. **Yapım (construction):** Kodlama, mühendislik testi denen birim testi, kod üzerinde test, sonra kullanıcı testi.
6. **Teslim ve müşteri değerlendirmesi:** Ürün kullanıcının kullanımına sunulur ve değerlendirmeler alınır. Bakım yapılıyorsa ardından yeni bir süreç başlar.

Hoca bu adımları çevrimler üzerinden bir kez daha izledi. Önce prototip yapılır, ama prototip oluşturulurken de müşterinin ne istediği dinlenir, prototip için plan yapılır, plan üzerinde risk analizi yapılır, analiz ve tasarım yapılır, yapım aşamasında prototipin kodu oluşturulur ve prototipin müşteri tarafından değerlendirilmesi istenir. İkinci çevrimde yeni ürün için müşteri

istekleri alınır, kullanılacak ürünün planı yapılır, plan üzerinde risk analizi yapılır, tasarım yapılır. Sonra bu kez gerçek kod oluşturulur. Önceki çevrim prototipti, bu çevrimde kullanılacak ürün kodlanır, test edilir ve kullanıma sunulur. Kullanıma hazırlık aşamasında girdi veri tabanlarına ilk bilgiler yerleştirilir ve kullanıcılar eğitilir.

7.3 Değerlendirme

Spiral modelin hem iteratif hem evrimsel özelliği vardır. Hoca onu şelale (ardışıl) modeli, prototip modeli ve artımlı modelin birleşimi olarak tanımladı.

Spiral modelde yazılım mühendisliğinde gerekli olan bütün dokümantasyonun yapılması şart koşulur. Bu yüzden günümüzde rekabete dayalı, hızlı yazılım çıkarmak isteyen küçük ya da büyük bir yazılım evi spiral modeli pek kullanamaz.

> **Hocanın vurgusu:** Spiral modelin kullanımı, tekrar tekrar kontrol yapılması ve çok fazla belge üretilmesiyle eş değerdir. Bu yüzden büyük ve özellikle hata kritik (safety-critical) projelerde kullanılır.

Hoca ardından staj deneyimine değindi: Bir yazılım firmasında bulunan öğrenciler, çoğunlukla çevik yöntemlerin artık yaygınlaştığını görmüştür. Özellikle daha küçük, daha hızlı yazılım geliştirme gayretinde olan yazılım evlerinde. Kurumsal firmalar ise kendilerine özgü çevik yaklaşımlar kullanıyor.

8. RAD (Rapid Application Development) Modeli

Hoca RAD modelini, IBM'in uzun yıllar önce önerdiği bir yazılım geliştirme süreç modeli olarak tanıttı. Hızlı yazılım ihtiyacı olan kurumlar için kullanılır. Kısa geliştirme çevrimlerine dayanan artımsal bir modeldir.

Ön koşulları şunlardır:

- Uygulama, kısa sürede geliştirilebilecek, birbirinden görece bağımsız modüllere ayrılabilmelidir. Önce parçalara bölünebilir olup olmadığına bakılır.
- Modüller eş zamanlı ilerleyebilmeli ve uygulama bileşen mimarisine uygun olmalıdır.
- Kurumda, aynı anda çalışabilecek her modül için ayrı ekipler bulunabilmelidir. Eş zamanlı çalışacak ekipler sağlanabiliyor mu sorusu sorulmalıdır.

Çok yüksek teknik riskler varsa RAD uygun değildir. Sürece uygun projelerde verimlilik artar ve yazılım çok kısa sürede geliştirilir.

RAD'ın temel özelliği şudur: İş modüllere ayrılır, her modül aynı anda ayrı bir ekip tarafından geliştirilir, sonra ekiplerin geliştirdiği modüller birleştirilir. Artımlı modelden farkı budur: Artımlı modelde modüller sırayla gelir, RAD'da aynı anda farklı ekiplerce geliştirilir. Genellikle iş modelleri tanımlanır ve dördüncü kuşak hızlı uygulama geliştirme araçları kullanılır.

> **Ders sonrası not:** Hoca RAD'ı "IBM'in önerdiği model" olarak tanıttı. RAD büyük olasılıkla James Martin'in IBM'deki çalışmalarına dayanır ve 1991'de onun kitabıyla yaygınlaşmıştır. Altyazıda modelin adı "RAT model" olarak geçiyor; doğrusu RAD'dır.

9. Bileşen Tabanlı (Component-Based) Geliştirme

Aynı hız fikri bileşen tabanlı uygulama geliştirmede de görülür. Bu yaklaşım, bileşen mimarisi uygulanan ortamlarda kullanılır. Sorulan sorular şunlardır: Konu alanında (domain) bir uygulama var mı? Bileşenler nasıl sınıflandırılacak, nasıl seçilecek? Genellikle yazılım evlerinin kendi bileşen kütüphanesi ya da veri tabanı vardır.

Adımlar şöyledir:

1. Önce konu alanı anlaşılır.
2. Hangi bileşenlerin bulunduğuna bakılır ve uygun olanlar seçilir.
3. Seçilen bileşenler yazılıma uyarlanır ve bir araya getirilir.

Uygun bileşenlerin bulunabilmesi için yazılım evinin zengin bir kütüphanesi olmalıdır. Bu model yeniden kullanımı (reusability) özendirir. Yeniden kullanım, yazılımın kendisinin, parçalarının, modüllerinin ya da çözüm yolunun yeniden kullanılabilmesidir. Yazılım çok emek isteyen

bir iş olduğu için reusability önemli bir kavram haline geldi. Hocaya göre bu konu eskiden beri uğraşılan bir şeydi, ama özellikle son 15 yılda kullanımı çok önem kazandı. Bileşen tabanlı yaklaşım, yeniden kullanıma özendiren ve hızlı uygulama geliştirmeyi sağlayan bir süreç modelidir.

Bileşen tabanlı yaklaşım RAD'da da görülür. Hızlı kurulan RAD ekipleri bileşen mimarisini uygulayarak daha hızlı yazılım üretebilir.

10. Birleşik Süreç (Unified Process)

Rational Unified Process, daha önceki yıllarda Rational firması tarafından çıkarılmış, ilk bütünleşik CASE araçlarından biridir. Başarılı olduğu görülünce IBM onu satın aldı. Bugün IBM'de bir platform düzeyinde çalışıyor ve yazılım geliştirmenin her aşamasında geliştiricilere destek sağlıyor. Bu aracın altında kullanılan modelin özellikle nesneye yönelik yazılım geliştirmeye uygun olduğu bilinir.

Spiral modelde önce müşteriyle iletişim, planlama, risk analizi, sonra modelleme, yapım ve teslim vardı. Birleşik süreç bunları fazlarda birleştirir:

Faz	Yapılanlar
Başlangıç (Inception)	Yazılım geliştirme fikrinin vizyonu. Başlangıç use case modeli (kullanıcı hikâyeleri). Başlangıç proje sözlüğü (konuyla ilgili kavramlar). İş süreci. Risk durumu.
Detaylandırma (Elaboration)	Müşteriyle iletişim ve planlama eş güdümlü yürür. Use case modelinin tanımlanması, analiz, yazılım mimarisinin oluşturulması, ön tasarım, risk listesi ve proje planının son hali. Plan sürekli revize edilir.
İnşa (Construction)	Tasarım modelinin çıkarılması. Yazılımın artımlarla bütünleşik olarak hazırlanması. Test planı, test case'ler, destek dokümantasyonu. Kullanıcı el kitapları ve kurulum açıklamaları.
Geçiş (Transition)	Teslim (deployment) ve sürüm (release): hem kullanıma hazırlık süreci hem de ürünün kullanıma verilmesi süreci.

Hocaya göre detaylandırma fazı, gereksinim analiziyle proje planını birleştirir: Yalnız müşteriyle görüşme değil, görüşülenleri plana aktarma da buradadır.

Release kelimesi ve sürüm yönetimi (release management) birleşik süreçle önem kazanmaya başladı. Bugün yazılım mühendisliğindeki önemli konulardan biridir: Yazılım ürününü kullanıma nasıl alacağız, bu aşamada hangi işlemler yapılır?

Hoca birleşik süreci, çevik süreçlere bir geçiş hazırlayan model olarak niteledi. Bütün aşamalarda dokümantasyon vardır, ama spiral modeldeki gibi tekrar tekrar döngüler yoktur. Bu da artımlı bir modeldir.

> **Ders sonrası not:** Hoca detaylandırma (elaboration) fazı için "başlangıç ve hazırlık aşaması" dedi. Başlangıç (inception) ayrı bir fazdır; elaboration "detaylandırma" ya da "ayrıntılandırma" diye çevrilir. Rational Software büyük olasılıkla 2003'te IBM tarafından satın alınmıştır.

11. Çevik (Agile) Yaklaşım

11.1 Ortaya Çıkışı

Çevik yaklaşımlara geçiş 2000'li yılların başındadır. O yıllarda yazılım mühendisliği gerekleri, belgeleme, standartların ve süreç modellerinin kullanımı yaygınlaşmıştı ve yazılım geliştiriciler belgeler ve standartlar arasında biraz fazla boğuldu. Bu bir manifesto doğurdu. Hocanın

sonraki toparlamadaki ifadesiyle manifestonun fikri şuydu: “Bizim işimiz kısa sürede yazılım elde etmek, yavaşlatan etkenleri gözden geçirmeliyiz.” Bunun karşılığında yeni modeller üretilmeye başlandı. Çevik süreçler yaklaşık 20 yıldan fazla bir süre önce kullanılmaya başladı.

Türkiye’de bu kavramlar 2003’te geldi. Hoca o dönemi şöyle anlattı: Herkes çevik sürecin ne olduğunu, nasıl yürütüldüğünü, Scrum’da nasıl bir çalışma sistemi olduğunu, proje koçlarının ve çevik koçların nasıl çalıştığını konuşuyordu. Hep süreç tanımı üzerinde duruluyordu, uygulayan yoktu. Konferanslardaki yayınlara bakıldığında da hep nasıl yapılacağı anlatılıyordu. Şirketlerdeki kullanım 2003’ten sonra başladı.

> **Ek bilgi (derste anlatılmadı):** Çevik Yazılım Geliştirme Manifestosu (Agile Manifesto) büyük olasılıkla 2001’de yayımlanmıştır. Manifestoda 12 ilkenin üstünde 4 temel değer vardır: süreçler ve araçlar yerine bireyler ve etkileşimler; kapsamlı belgeler yerine çalışan yazılım; sözleşme pazarlığı yerine müşteriyle işbirliği; plana bağlı kalmak yerine değişime yanıt vermek. Manifesto sağdakilerin değersiz olduğunu değil, soldakilere daha çok değer verildiğini söyler. Hocanın aşağıdaki 4 temel fikri bu değerlerle örtüşen bir yorumdur.

11.2 Çevik İlkeler

Hoca çevik yaklaşımın hangi kurallara bağlı olduğunu slayttaki maddeler üzerinden anlattı:

- Müşteri memnuniyetini daha erken almak. Müşteriye daha erken aşamalarda ulaşmak. Kullanılabilir yazılımı müşteriye daha erken aktarmak.
- Gereksinimlerin daha hızlı aktarılması. Bu, müşteriyle daha sıkı ve yakından çalışma, müşteri temsilcisini ekibe dahil etme fikrini getirdi.
- Yazılımın artımlı olarak müşteriye daha hızlı aktarılması. Artımlı modeller zaten vardı, ama onlara hız kazandırma yoluna gidildi.
- Yazılımı kullanacak kişiler ve geliştiriciler projenin içinde, günlük hayatta birlikte çalışır. Hoca, kullanıcıların bir işletmeden olmasının şart olmadığını söyledi.
- Projede çalışanları daha fazla motive edecek bir ortam. Kişilere görevler verilir ve herkesin birbirini kontrol edeceği şekilde değerlendirilir.
- Geliştirme ekibinin yüz yüze toplantıları daha hızlı yapması. Diğer süreç modellerinde rapor hazırlanır, rapor üzerinden inceleme yapılır, düzeltmeler diğer ilgililere aktarılır. Çevik yaklaşımda bunun yerine yüz yüze görüşmeler vardır.
- Çalışan yazılımın her süreç sonunda değerlendirilmesi ve ölçülmesi. Bu değerlendirme, çalışan yazılımın daha hızlı aktarılmasıyla mümkün olur.
- Çevik süreçlerin sürdürülebilir geliştirmeyi canlandırdığı belirtilir.
- Sponsorlar, geliştiriciler ve kullanıcıların tanımlı olması ve birbirini daha fazla kontrol edebilmesi. Sponsor, yazılım geliştirme işinde parayı ödeyen, finansmanı sağlayan kişidir. Hoca uç kullanıcının her zaman başka, parayı veren yönetimin başka olduğunu hatırlattı.
- Tasarım aşamasının teknik olarak daha iyi ve daha çevik olması.
- Basitlik: Gereksinim analizi sırasında çok karmaşık çözümleri art arda belirtmek yerine basit çözümü söylemek, onun geliştirilmesi ve değerlendirilmesi üzerinden gitmek.
- Ekibin düzenli aralıklarla birbirini kontrol etmesi ve hem davranış hem teslim aşamasında kendi kendini kontrol eden bir yapının olması.

11.3 Hocanın Özetlediği 4 Temel Fikir

Hoca, çevik ilkelerin 12 maddede özetlendiğini, ama bunların altında 4 temel iş gördüğünü söyledi:

1. Müşteri üretilen kodu daha hızlı görür ve değerlendirmeyi bu kodlar üzerinden yapar.
2. Müşteri ekibin içinde olacak kadar yakındır. Çalışmayı anında kontrol eder, sonuçları anında değerlendirir.
3. Ekip içinde daha dinamik bir yapı oluşturulur.
4. Basitlik: Çok karmaşık modelleri daha gereksinim analizinde belirlemek yerine, basitten karmaşığa doğru bir yol izlenir.

12. Scrum ve Diğer Modellerin Karşılaştırılması

Hoca bir slaytta şelale, spiral, iteratif modeller ve Scrum’ın karşılaştırıldığı bir tablo gösterdi:

Kriter	Waterfall	Spiral	Iterative	Scrum
Tanımlı süreçler	Gerekli	Gerekli	Gerekli	Planlama ve kapatma aşamasında
Final ürün	Planlamada tanımlanmış	Planlamada tanımlanmış	Proje esnasında şekilleniyor	Proje esnasında şekilleniyor
Proje maliyeti	Planlama esnasında hesaplanıyor	Kısmen değişken	Proje esnasında şekilleniyor	Proje esnasında şekilleniyor
Tamamlanma tarihi	Planlama esnasında belirleniyor	Kısmen değişken	Proje esnasında şekilleniyor	Proje esnasında şekilleniyor
Dış çevreye duyarlılık	Sadece planlamada	Planlama öncelikli	İterasyon sonlarında	Her zaman
Takım esnekliği, yaratıcılığı	Limitli	Limitli	Limitli	İterasyonlar boyunca limit yok
Bilirkişi desteği (harici)	Proje eğitimi esnasında	Proje eğitimi esnasında	Proje eğitimi esnasında	Gerekirse proje içinde takıma dahil edilebilir
Başarı olasılığı	Düşük	Orta düşüklükte	Orta seviye	Yüksek

> **Hocanın vurgusu:** Tablodaki başarı olasılıkları Scrum tarafından bakılarak yazılmıştır. "Çevik her projede mutlaka başarılıdır" diye bir şey yoktur. Hangi işin, hangi ekiple, hangi konuda yapılacağına ve yazılımın özelliğine bakılarak, yeni başlanacak projede model buna göre seçilmelidir. Süreç modeli projenin tipine, ekibin yapısına ve yazılımın konusuna göre belirlenir.

13. Uç Programlama (XP, Extreme Programming)

Hoca XP'yi ilk kullanılan çevik model olarak tanıttı. Adı "extreme" kelimesinin X'i ile "programming" kelimesinin P'sinden gelir. Hoca, 2003'ten itibaren yazılarında buna "aykırı programlama", "uç programlama" gibi değişik Türkçe adlar verdiklerini, ama hiçbirinin tam oturmadığını söyledi. Bugün de kısaltması olan XP kullanılıyor.

> **Ders sonrası not:** Hoca XP'yi "ilk çevik model" olarak andı. Scrum büyük olasılıkla 1995'te, XP ise 1996-1999 arasında Kent Beck tarafından ortaya konmuştur. İkisi de manifestodan öncedir; XP'yi tek başına "ilk" saymak tartışmalıdır.

XP artımlı bir modeldir. Döngü şöyledir: planlama → tasarım → kodlama → test → artımsal sürüm (release). Hocaya göre önemli olan, bu adımların her birinde XP'de neyin farklı yapıldığıdır. Hoca XP'yi aranın öncesinde ve sonrasında iki kez anlattı. Aşağıda iki anlatım birleştirildi.

13.1 Planlama

- **Kullanıcı hikâyeleri (user stories):** Planlama sırasında müşteriden kullanıcı hikâyeleri alınır. Her kullanıcıdan işi nasıl yaptığına ilişkin iş akışı anlaşılmaya çalışılır. Hocanın örneği bir satış sipariş ve fatura hazırlama işiydi. Satışta, kasada bulunan kişi işi kendi odaklı anlatır. Stokta bulunan kişi kendi yaptıkları odaklı anlatır. Bunlar farklı bakış açılarıdır. Farklı kişilerin yaptığı farklı işler, farklı kullanıcı hikâyeleriyle dinlenir. Bu yüzden "kullanıcı hikâyesi" denir.
- **Kabul testi kriterleri (acceptance test criteria):** Her kullanıcının yaptığı işte ne istediği, neyin özellikle kontrol edilmesini istediği sorulur. Yani yazılım bittiğinde özellikle neyin test edileceği baştan belirlenir. Test case'ler planlama sırasında ortaya çıkar.
- **İterasyon planı:** Hangi modül önce geliştirilmeli, ne yapılmalı?

Test güdümlü geliştirme (TDD, Test-Driven Development): Hoca kabul testi kriterlerinin baştan alınmasını test odaklı, test yönelimli geliştirme ile ilişkilendirdi. Bu yaklaşım Scrum'da da vardır, aslında bütün çevik modellerde bulunur. Hocaya göre TDD bir süreç modeli değildir, çevik modellerin çoğunun içinde yer alan bir yaklaşımdır.

13.2 Tasarım

- **Basit tasarım:** Çevik yaklaşımda basitlik önemli olduğu için önce basit bir tasarım yapılır. Kullanıcı hikâyelerine göre modüllerdeki hangi nesnelerin nelerle ilişkili olduğu belirlenir.
- **CRC kartları:** Her sınıf için bir kart hazırlanır. Kartta sınıfın tanımı (Class), sorumluluğu (Responsibility), yani hangi işleri yapmakla sorumlu olduğu, ve bu işleri gerçekleştirirken hangi sınıflarla işbirliği yapacağı yazılır. Hoca bunları "sınıf, sorumluluk ve işbirliği kartları" olarak da andı.
- **Prototipler:** Tasarım için çözüm önerileri ve bunlar için prototipler hazırlanır.

  Ders sonrası not: Hoca CRC'nin üçüncü harfi için "collaboration" ve "collaboration kartlarının oluşturulması" dedi. CRC'nin C'si büyük olasılıkla "Collaborator"dır: Kartın bir bölümüne sınıfın işbirliği yaptığı diğer sınıflar yazılır. Ayrı bir "collaboration kartı" yoktur, aynı kartın bir bölümüdür.

13.3 Kodlama

- **İkili programlama (pair programming):** Hoca bunun bütün çevik modellerde olmadığını, XP'nin bir özelliği olduğunu, örneğin Scrum'da bulunmadığını vurguladı. Bir yazılım parçasını geliştirmekle sorumlu geliştirici tek başına değil, iki kişi çalışır. Biri kodu yazarken diğeri başka bir gözle o kodu test etmiş olur.
- **Yeniden düzenleme (refactoring):** Eski kodlarda düzenleme yapma, kod üzerinde iyileştirme işlemleri. Bu da XP'de olması gereken bir şeydir.

  Hocanın vurgusu: İkili programlama XP'ye özgüdür, Scrum'da yoktur.

  Ek bilgi (derste anlatılmadı): İkili programlamada iki kişinin rolü büyük olasılıkla şöyle adlandırılır: kodu yazan "sürücü" (driver) ve kodu anlık gözden geçirip yön veren "yön bulucu" (navigator). Roller sık sık değişir. Hocanın "diğeri test eder" dediği, bu anlık gözden geçirme rolüdür. TDD'nin klasik tanımında ise testler koddan önce yazılır: önce başarısız olan bir test yazılır, sonra onu geçecek kadar kod yazılır, sonra yeniden düzenleme yapılır (kırmızı-yeşil-yeniden düzenle döngüsü).

13.4 Test

- **Birim testi:** Kodu yazılan her program parçasına birim testi uygulanır.
- **Sürekli bütünleştirme (continuous integration):** Birim testi yapıldıktan sonra her birim bittikçe bütündeki yerine konur ve bütünlük testi yapılır.
- **Kabul testi:** Planlamada (gereksinim analizinin başında) alınan kabul testi kriterlerine göre yapılır.
- **Müşterinin denemesi ve sürüm:** Müşteri artımsal ürünü dener. Parça bitince yeni bir yazılım artımı ortaya çıkar. Her artım release edilir, yani dışarıdaki kullanıcılara teslim edilip kullanıma alınır. Hoca, bu son aşamanın yalnız sınama değil, uygulamaya geçiş süreci olduğunu söyledi.
- **Proje hızı:** Her release sonunda projenin plandaki hızının sağlanıp sağlanmadığı hesaplanır.

Hoca test aşamalarının sonraki derslerde ayrıntılı görüleceğini söyledi. Sıra şöyledir: birim testi → bütünlük testi → sistem testi → kullanıcı testi.

13.5 XP'nin Özeti

Hoca XP'ye özgü noktaları aklınızda kalsın diye bir kez daha sıraladı:

- **Planlama:** Kullanıcı hikâyeleri, kabul testi kriterlerinin baştan verilmesi (test güdümlü geliştirme), iterasyon planı.
- **Tasarım:** Basit tasarım. Her sınıfın hangi işi yaptığı, hangi sorumlulukta olduğu ve hangi sınıflarla işbirliği yaptığı (CRC kartları). Prototip oluşturulması.
- **Kodlama:** İkili programlama ve yeniden düzenleme.
- **Test:** Birim testlerinin ve sürekli bütünlük testlerinin sürekliliği, planlamadaki kabul testlerinin yapılması, ardından sürüm.

14. Scrum

Hoca aranın ardından çevik modeli bir kez daha konumlandırdı: “Çevik model” tek bir modelin adı değildir. Çevik yaklaşıma uygun yazılım geliştirme süreç modelleri vardır ve önce bunların ortak özellikleri anlatılmıştır. Klasik modeller tarafında şelale, prototip, RAD, bileşen tabanlı, spiral, artımlı ve birleşik süreç görüldü. Birleşik süreç, nesneye yönelik yaklaşımın yerleşmesiyle çevik modellere bir geçiş modelidir.

14.1 Adın Kökeni

Hocaya göre Scrum kelime anlamı olarak, Amerikan futbolunda aralarda oyuncuların sürekli yaptığı taktik geliştirme toplantısıdır. Fikir şudur: Yazılım geliştirme de bir zafer kazanma koşusudur, öyleyse bunu taklit eden bir yaklaşım geliştirelim.

> **Ders sonrası not:** Hoca Scrum’ın kökenini Amerikan futbolu olarak verdi. Scrum terimi büyük olasılıkla ragbiden (rugby) gelir: Oyunu yeniden başlatmak için oyuncuların kenetlenerek oluşturduğu toplu düzenin adıdır.

14.2 İşleyiş

1. Bir görev listesi çıkarılır ve koşu bu listeye göre başlar. Önce yapılacak iş alt parçalara, alt görevlere bölünür. Görevler aylık, haftalık, günlük olarak gruplanır.
2. Görev listesi kullanıcı hikâyelerinden ortaya çıkar. Bunlar önceliklendirilir: Hangi kullanıcı hikâyesi daha önce yazılmalı?
3. Koşuda (sprint) görev listesinin maddelerinden biri seçilir ve bir ile dört hafta arasında gerçekleştirilir.
4. Koşu boyunca ekip her gün kısa, 15 dakikalık bir toplantı yapar. Sonuçlar gösterilir, grubun hepsi sonuçları görür.

Toplantıyı yöneten kişiye hoca “proje lideri”, “proje koçu” ya da “çevik koç” denebileceğini söyledi. Bu kişi toplantıyı yönlendirir. Toplantılarda cevaplanmaya çalışılan üç temel soru vardır:

1. Son toplantıdan bugüne kadar ne yaptınız?
2. Karşılaştığınız engeller nelerdir? Hoca bunu bir bilgi birikimi (know-how) oluşturma biçimi olarak gördü.
3. Yarınki toplantıya kadar neleri başarmayı hedefliyorsunuz?

Scrum panosu (Scrum board): Meşhur bir tahtadır. Üzerinde hangi işlerin yapılacağı, hangilerinin yapıldığı, hangilerinin yapılmadığı işaretlenir. Böylece proje yöneticisinin tek tek sormasına gerek kalmaz: Hangi iş yapıldı, hangisi yapılmadı, kim yaptı, kim yapmadı görülür. Herkes birbirini denetlemiş olur.

Müşteri de ekip içinde birlikte çalışan kişilerden biridir. Onun görevi yapılanları değerlendirmek ve kontrol etmektir.

14.3 Karşılaştırma Tablosuna Dönüş

Hoca Scrum’ın işleyişini anlattıktan sonra ilk gösterdiği tablonun satırlarını bu işleyişle açıkladı:

- **Final ürün:** Son ürün proje esnasında şekillenir.
- **Tanımlı süreçler:** Şelale ve spiralde süreçler tanımlıdır. Scrum’da planlama ve kapatma aşamaları arasındaki işlemler sürekli gözden geçirilir.
- **Proje maliyeti:** İlk başta kabaca bir maliyet belirlenir, ama plan zaten sürekli güncellenir.
- **Takım esnekliği ve yaratıcılığı:** İterasyonlar boyunca çalışanların kendi fikirlerini söylemesi, ekip yaratıcılığını ve esnekliğini sağlar.
- **Bilirkişi desteği:** Diğer modellerde danışmanlar yalnız proje eğitiminde kullanılır. Scrum’da danışmanlar proje içinde ekibin bir üyesi olarak da çalışır.

  > **Ek bilgi (derste anlatılmadı):** Scrum Kılavuzu’nda (Scrum Guide) büyük olasılıkla üç rol tanımlanır: Scrum Master, Product Owner ve geliştiriciler. Hocanın “proje lideri / proje koçu / çevik koç” dediği kişi Scrum Master’a, ekipteki müşteri temsilcisi

Product Owner'a karşılık gelir. Görev listesinin adı Product Backlog'dur; bir koşuya seçilen maddeler Sprint Backlog'u oluşturur. Günlük 15 dakikalık toplantıya Daily Scrum denir.

15. Model Seçimi ve Kuruma Özgü Uyarlama

Hoca yazılım geliştirme süreç modellerinin hemen hemen tümünün görüldüğünü söyledi. Bunların dışında Kanban da bir çevik süreçtir. Bu derste üzerinde durulmadı, gerekirse daha sonra durulabilir.

Toparlarsak: Klasik süreç modelleri görüldü. Ardından çevik ilkelerin ne olduğu ve çevik yaklaşım içinde XP ile Scrum işlendi.

Bugün Scrum en çok kullanılan çevik modeldir. Önce yalnız küçük yazılım evleri, kendilerine bir dinamizm kazandırmak için kullanmaya başladı. Daha sonra büyük firmalarda birçok projede, hatta askeri yazılımlarda bile Scrum kullanıldığı görüldü. Hoca bunun, spiral modelin çok uygun olduğu hata kritik yazılımlarda bile görüldüğünü vurguladı.

Yazılım mühendisliği sempozyumlarında tartışılan konulardan biri şudur: Scrum yaygınlaştı, ama gerçekte ne kadar fayda sağladı? Diğer klasik süreçlerle farkı ne? Hangi iş hangisiyle daha iyi yapılır? Bu tartışmalar sürüyor.

Bazı kurumlar Scrum'ın aynısını almıyor. Birçok projesinde şelale kullanan kurumlar, hem ekibi hem projeleri daha iyi uyarlamak için kendilerine özgü bir çevik model tanımlıyor. Hoca örnek olarak büyük bir Türk kurumunun 5-6 yıl önce kendine uyarladığı bir çevik modeli andı.

Hocaya göre önemli olan şudur: Ekip kendini disipline etmeli ve gereksinimlerdeki müşteri isteklerini gözden kaçırmamalıdır. Tasarım aşamasında uygun modelleri kurabilmelidir. Kurum "şu modeli seçtim, aynısını uygulayacağım" demek yerine, süreklilik sağlayacak ve bütün adımları sürecin içine alacak bir yapı tanımlamalıdır. Bu yapı analizi, tasarımı, kodlamayı ve testi şemsiye aktiviteleriyle birlikte kapsamalı, kurumun ve projelerin kendisine en uygun yapı olmalıdır. Sonra kurum bu modele uymalıdır.

> **Ek bilgi (derste anlatılmadı):** Derste anlatılan klasik modellerin kökenleri büyük olasılıkla şöyledir: Şelale modeli genellikle Winston Royce'un 1970 tarihli makalesine dayandırılır; Royce aslında yinelemesiz şelalenin riskli olduğunu da yazmıştır. Spiral model Barry Boehm tarafından 1986-1988 yıllarında önerilmiştir ve risk odaklı oluşu buradan gelir.

Örnek Sorular

Bu derste hazır kare ve video yoktur. Aşağıdaki sorular hocanın derste sözlü sorduğu sorulardır.

Soru 1 (derste sözlü, 00:10:33): Yazılım dediğimiz zaman programdan başka neleri de düşünüyoruz? Yazılım sadece program mıdır?

Derste verilen cevap: Hayır. Öğrenciler dokümantasyon, belgeler dedi. Hoca şöyle tamamladı: Kullanıcıya yazılımın nasıl kullanılacağını anlatan kullanıcı belgeleri, bakım yapacaklara iç yapıyı ve veri yapısını anlatan belgeler, yazılımın yapım aşamasını anlatan belgeler ve veri tabanının hangi test verisiyle nasıl test edildiğini anlatan belgeler. Bunların hepsi programla birlikte yazılımı oluşturur.

Soru 2 (derste sözlü, 02:25:04 ve 02:25:55): Çevik süreçlerde 12 madde saydık, ama 4 temel fikir vardı. Çevik süreçlerin dayanağı, temeli nedir? Çevik süreçler neden ortaya çıkmıştır? Klasik süreçlerde de analiz, tasarım, kodlama ve test var; çevik süreçlerin farkı nedir?

Öğrencilerin cevapları eksik kaldı. Bir öğrenci klasik süreç adımlarını (gereksinim belgesi, analiz, tasarım, kodlama, test, ürün) saydı; hoca bunların klasik süreçlerde de bulunduğunu söyledi. Başka bir öğrenci "etkileşim halinde olması" ve "belli bir planı izlemek" dedi. Hoca cevabı vermeden dersi kapattı.

> **Çözüm (derste verilmedi):** Derste anlatılanlara göre çevik süreçler 2000'li yılların başında, yazılım geliştiricilerin belgeler ve standartlar arasında boğulması üzerine yayımlanan bir manifestoyla ortaya çıktı. Temel fikir, kısa sürede yazılım elde etmek

ve süreci yavaşlatan etkenleri ortadan kaldırmaktır. Hocanın 4 temel fikri şunlardır: (1) Müşteri üretilen kodu, çalışan yazılımı daha hızlı görür ve değerlendirmeyi onun üzerinden yapar. (2) Müşteri ekibin içindedir, sonuçları anında değerlendirir. (3) Ekip içinde dinamik bir yapı vardır; raporlar yerine yüz yüze görüşmeler yapılır. (4) Basitlik: Basit çözümle başlanıp basitten karmaşığa doğru ilerlenir. Öğrencinin söylediği "belli bir planı izlemek" çevik yaklaşımın tersidir: Manifestonun değerlerinden biri büyük olasılıkla "plana bağlı kalmak yerine değişime yanıt vermek"tir. Scrum'da da plan sürekli güncellenir ve son ürün proje esnasında şekillenir.

Dersin Toparlaması

Bu derste önce geçen dersin konuları tekrar edildi: yazılımın yalnız program olmadığı, yazılım türleri, eski yazılım, müşteri, yönetim ve geliştirici açısından yanılgılar, yazılım mühendisliğinin tanımı, temel aktiviteler, şemsiye aktiviteleri ve CASE. Ardından yazılım mühendisliğinin beslendiği bilim dalları ve dersin hedefleri gözden geçirildi. Asıl konu yazılım geliştirme süreç modelleriydi. Hocanın ana fikri, temel aktivitelerin her zaman sırayla gitmediği ve adımların nasıl dizildiğinin modeli belirlediğiydi. Klasik modeller olarak şelale, prototip, artımlı, spiral, RAD, bileşen tabanlı model ve birleşik süreç görüldü. Şelalenin en büyük zayıflığı müşterinin çalışan ürünü en sonda görmesiydi. Spiral model çok belge ve risk analizi gerektirdiği için hata kritik projelere uygun bulundu. Birleşik süreç ise çevik modellere geçiş modeli olarak konumlandı.

Sonra çevik yaklaşımın neden doğduğu, ilkeleri ve hocanın bunları topladığı 4 temel fikir anlatıldı. Çevik modellerden XP planlama, tasarım, kodlama ve test adımlarındaki kendine özgü uygulamalarıyla (kullanıcı hikâyeleri, kabul testi kriterleri, CRC kartları, ikili programlama, yeniden düzenleme, sürekli bütünleştirme), Scrum da koşu, günlük toplantı ve Scrum panosuyla işlendi. Hoca hiçbir modelin her projede başarılı olmadığını, kurumların projelerine, ekiplerine ve konularına göre kendilerine uygun bir model tanımlayıp ona uyması gerektiğini vurguladı. Önümüzdeki hafta yazılım kurumlarında sertifikasyon (CMM ve CMMI, yani yazılım evinin yeterliliği) ele alınacak, geliştirme sürecindeki adımlara açıklık getirilecek ve ardından gereksinim analizine geçilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.